ЧАСТЬ 2 · КУРС PERL

Perl: управление потоком, ошибки и модули

```
if ($valid) {
    process_data();
} else {
    die "Invalid input";
}
```

Новые языки программирования · Часть 2

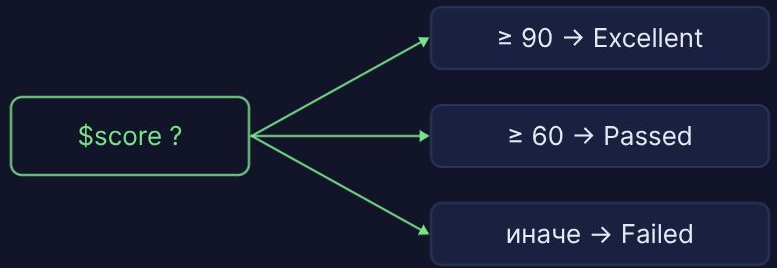
04 Поток · 05 Ошибки · 06 Модули

Условия определяют ход программы

```
my $score = 85;

if ($score >= 90) {
    print "Excellent\n";
} elsif ($score >= 60) {
    print "Passed\n";
} else {
    print "Failed\n";
}
```

- if и elsif проверяют несколько условий по очереди
- else задаёт альтернативную ветвь
- Условие — в круглых скобках, блок — в фигурных (скобки обязательны)



Короткие условия и множественный выбор

- Условие можно записать после оператора: действие if условие
- unless выполняет код, если условие ложно
- Тернарный оператор ?: выбирает одно из двух значений
- given/when — экспериментальные и устаревшие (с Perl 5.38 вызывают предупреждения); не универсальная замена switch

```
print "Adult\n" if $age >= 18;

die "Invalid value\n"
    unless $value > 0;

my $message = $status eq "ok"
    ? "Success" : "Error";
```

Модификаторы удобны для простых однострочных действий. Сложную логику лучше записывать обычными блоками if / else.

Повторение действий

- while выполняется, пока условие истинно; until — пока условие ложно
- for и foreach перебирают последовательности и управляют повторениями

while — проверка перед итерацией

```
my $i = 1;

while ($i <= 3) {
    print "$i\n";
    $i++;
}
```

Условие → тело → снова условие. Счётчик меняем вручную.

for — перебор диапазона

```
for my $n (1 .. 3) {
    print "$n\n";
}
```

Оператор .. строит список 1, 2, 3; $n получает каждое значение.

foreach: обработка элементов по очереди

```
my @numbers = (1, 2, 3, 4, 5);

foreach my $n (@numbers) {
    next if $n % 2 == 0;
    print "$n\n";
}
```

- foreach перебирает элементы списка или массива
- Переменная цикла получает текущий элемент
- last — выход из цикла, next — следующая итерация, redo — повтор текущей



next пропускает остаток тела цикла и сразу переходит к следующему элементу. Вывод: 1, 3, 5.

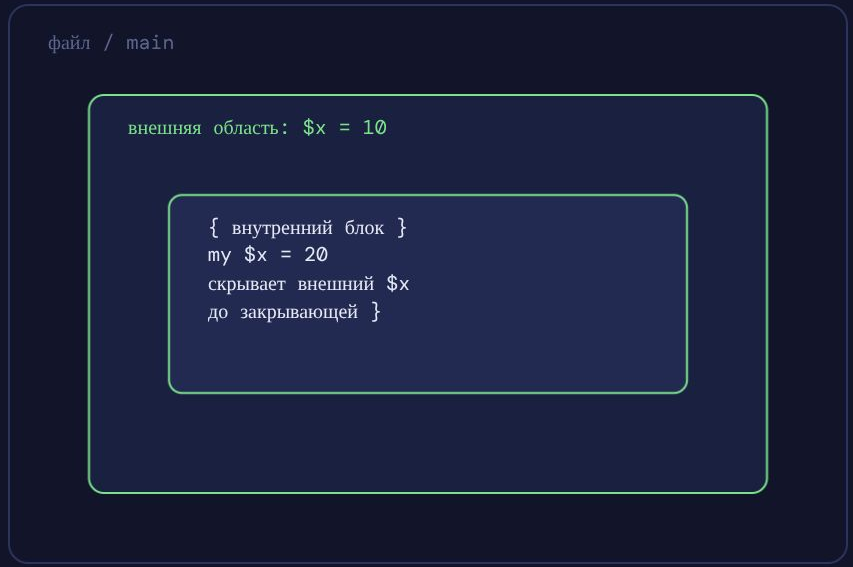
Фигурные скобки задают блок

```
my $x = 10;

{
    my $x = 20;
    print "$x\n";  # 20
}

print "$x\n";      # 10
```

- Блок { ... } объединяет операторы
- my-переменные видны только внутри своего блока
- Блоки — в условиях, циклах, обработчиках и для ограничения видимости



Как программа сообщает об ошибке

```
use strict;
use warnings;

my $file = "data.txt";

open my $fh, "<", $file
    or die "Cannot open $file: $!";

warn "File is empty\n" if -z $file;

close $fh;
```

- die выбрасывает исключение; без перехвата программа завершается
- warn выводит предупреждение в STDERR и не останавливает работу
- die — для критических ошибок и проверки предусловий

warn

Предупреждение. Выполнение продолжается.

die

Критическая ошибка. Выполнение прекращается, если исключение не перехвачено.

Перехват ошибки без немедленного завершения

```
my $result = eval {
    die "Calculation failed\n";
    42;
};

if ($@) {
    warn "Error: $@";
} else {
    print "$result\n";
}
```

- eval { ... } перехватывает исключения, возникшие внутри блока
- Текст исключения обычно доступен через $@
- После блока проверяем результат и обрабатываем ошибку
- eval не спасает от всех сбоев: например, сигналы и exit он не перехватывает



Проверяем предположения программы

- Проверки предусловий выявляют некорректные данные
- В ядре Perl нет универсального оператора assert
- Проверки пишем вручную через die, тесты — с Test::More
- Проверки и автотесты решают разные задачи

Проверка предусловия

```
sub divide {
    my ($dividend, $divisor) = @_;

    die "Division by zero\n"
        if $divisor == 0;

    return $dividend / $divisor;
}

print divide(10, 2), "\n";
```

Автоматический тест

```
use Test::More tests => 1;

is(divide(10, 2), 5, "division works");
```

Фрагмент теста: в полноценном тестовом файле (.t) функция divide должна быть определена или импортирована из модуля. Test::More входит в стандартную поставку Perl.

package: разделение имён

```
package MathTools;

sub square {
    my ($x) = @_;
    return $x * $x;
}

package main;

print MathTools::square(4), "\n";
```

- package объявляет пространство имён для переменных и подпрограмм
- Полное имя записывается через ::
- Пакеты исключают конфликты имён



Пакет ≠ файл: один файл Perl может содержать несколько объявлений package.

Модуль — код, который можно переиспользовать

- Модуль — обычно файл .pm с пакетом и его реализацией
- use загружает модуль на этапе компиляции и обычно вызывает его import
- require загружает файл во время выполнения и не вызывает import — это не полная замена use

MathTools.pm

```
package MathTools;

use strict;
use warnings;

sub square {
    my ($x) = @_;
    return $x * $x;
}

1;
```

main.pl

```
use strict;
use warnings;
use MathTools;

print MathTools::square(4), "\n";
```

Завершающее 1; возвращает истинное значение при загрузке файла — иначе use / require сообщат об ошибке. Файл MathTools.pm должен лежать в каталоге из @INC.

Как модуль предоставляет функции

MathTools.pm

```
package MathTools;

use strict;
use warnings;
use Exporter 'import';

our @EXPORT_OK = qw(square);

sub square {
    my ($x) = @_;
    return $x * $x;
}

1;
```

- Экспорт определяет, какие имена можно импортировать
- Exporter — распространённый механизм экспорта
- @EXPORT — импорт по умолчанию
- @EXPORT_OK — только по явному запросу
- Полное имя MathTools::square работает и без экспорта

main.pl

```
use MathTools qw(square);
print square(4), "\n";
```



СВЯЗЬ ТЕМ · 04 + 05 + 06

От отдельных конструкций к структуре программы

```
use strict;
use warnings;
use MathTools qw(square);    # модуль

my $input = $ARGV[0] // "";

my $result = eval {
    die "Not a number\n"    # проверка
        unless $input =~ /^-?\d+$/;
    square($input);
};

if ($@) { warn "Error: $@" }  # ошибка
else    { print "$result\n" }
```

- Условия и циклы определяют порядок выполнения
- die, eval и проверки управляют ошибочными ситуациями
- Пакеты и модули делят программу на независимые части
- Импорт делает интерфейс модуля удобным

ИТОГИ · ЧАСТЬ 2

Что важно в Perl?

01

УПРАВЛЕНИЕ

if, while, foreach задают логику выполнения

02

ОШИБКИ

die, warn, eval помогают реагировать на сбои

03

ПРОВЕРКИ

Предусловия и тесты (Test::More) выявляют проблемы

04

МОДУЛИ

package, use и экспорт помогают организовать код

Perl сочетает гибкие конструкции управления с возможностью делить программу на повторно используемые модули.